

WAS Reporting Code Base Analysis

July 2026

Prepared for modernization planning and Databricks compatibility assessment.

Introduction

This analysis documents the current Web Application Scanning reporting code found under backend/was. The goal is to identify major components, legacy dependencies, data flows, and integration points that must be understood before modernizing the reporting pipeline for Databricks compatibility.

The review is based on static inspection of the extracted source code. No external systems were called, and no report generation workflows were executed.

Changelog

Revision Log

Revision	Date	Approved By	Comments
0.1	07/16/2026	TBD	Initial analysis draft based on backend/was static review.

Dependencies

The WAS code base relies on a mix of local workstation state, Python libraries, Docker, LaTeX tooling, Qualys APIs, and AWS DynamoDB customer metadata. Dependency versions are mostly unpinned, except qualysapi.

Dependency Area	Evidence	Modernization Concern
Python runtime	Dockerfile uses python:latest and installs python3 plus texlive-xetex.	Unpinned runtime creates reproducibility and supply-chain risk. Databricks jobs should pin runtime, libraries, and OS/package dependencies.
Python packages	requirements.txt includes qualysapi==5.1.0, lxml, pandas, openpyxl, PyPDF4, pikepdf, pypdf, pdfrw, matplotlib, seaborn, pymongo, docker, pyarrow, and duplicates of openpyxl.	Dependency set is broad and includes unused or overlapping PDF libraries. Needs pruning and version pinning before migration.
Local configuration	was_config.txt under the user home directory supplies Qualys credentials, host, daily tracker path, report	Local config file should be replaced by managed configuration and secrets, such as AWS Secrets

	mailer path, log path, and special cases path.	Manager, Databricks secrets, or environment-specific config.
Docker daemon	reporting.py launches up to 30 local containers from image was_report_creator.	Databricks should not depend on a local Docker daemon for task fanout. Use Databricks Workflows, Jobs, or task-level parallelism.
LaTeX and PDF tooling	WAS_report_creator.py renders Mustache to .tex and runs xelatex twice, then encrypts PDFs.	Report rendering may require a containerized or job-scoped rendering service if Databricks cluster images do not support the full LaTeX/PDF stack.
AWS DynamoDB helper	Imports was_dynamodb.dynamodb and was_dynamodb.attribute for stakeholder data reads and updates.	The helper package is not included in backend/was. Its API and deployment source must be located before migration.

Objective

This report is intended to support the Jira user story by documenting the current WAS reporting pipeline and highlighting the changes needed to support a Databricks-compatible architecture.

Identify major WAS components and responsibilities.

Document inbound data, generated artifacts, and outbound handoffs.

Catalog integration points with Qualys, DynamoDB, Excel workbooks, local files, Docker, and other legacy systems.

Identify migration challenges and make practical modernization recommendations.

Scope and Authority

In scope: files under backend/was, including the top-level reporting orchestrator, the wasreport report generator, the update_tracker workflow, templates, and bundled static assets.

Out of scope for this draft: live Qualys behavior, actual customer data, production report outputs, AWS account configuration, CI/CD integration, and runtime validation against Databricks.

Important finding: static search did not identify a direct Redshift integration in backend/was. Redshift remains an acceptance-criteria item to validate against the broader repository and

runtime environment, but this WAS export itself appears to use Qualys, DynamoDB, Excel, local files, and Docker rather than Redshift directly.

Code Base Overview

Component	Primary Files	Responsibility
Batch report orchestrator	backend/was/reporting.py	Reads the daily reports workbook, skips running or completed rows, launches one Docker container per stakeholder tag, validates generated PDFs, copies reports to the report mailer folder, archives reports, and zips mailer contents.
Report generator CLI	backend/was/wasreport/ wasreport/ WAS_report_creator.py	Calls Qualys WAS APIs, creates XML and PDF reports, parses XML, builds CSV attachments and graphs, renders the final PDF through Mustache and LaTeX, optionally encrypts reports, and supports administrative Qualys actions.
Tracker updater	backend/was/ update_tracker/ update_tracker/main.py and utils	Queries Qualys schedules and scans, consolidates scan slices into tracker items, updates the daily Excel tracker, updates DynamoDB scan metadata, and can delete inaccessible web applications from Qualys.
Rendering templates and assets	NEW_BIG.mustache and assets/*	Provide the LaTeX report template, CISA branding assets, Qualys report request XML templates, fonts, cover pages, and images used in the final report.
PDF redaction support	redact_qualys.py and pdf_redactor.py	Redacts Qualys-generated detail PDFs before watermarking and attaching them to the generated report.

Data Flows

Daily Report Generation

Step	Input	Processing	Output or Side Effect
1	was_config.txt	Reads dailyReportsFilePath and reportMailerFilePath.	Local path configuration loaded at process startup.
2	Daily reports Excel workbook	Iterates column B for stakeholder tags; skips rows marked Running or already handled.	Selected stakeholder tags and per-row password settings.
3	Stakeholder tag and password	Launches was_report_creator Docker containers with tag and encryption password.	Detached local containers generate PDFs under the mounted home directory.
4	Generated stakeholder PDFs	Checks existence and encryption state using PyPDF4.	Only encrypted and readable PDFs are copied forward.
5	Report mailer folder	Copies PDFs to reportMailerFilePath, moves originals to ~/Reports, zips mailer folder, and deletes the individual files in that folder.	reportsMMDD.zip ready for downstream mailer or Teams workflow.

Single Report Generation

Step	Input	Processing	Output or Side Effect
1	Stakeholder tag	Looks up Qualys tag ID and web application count.	Determines whether to include a details PDF.
2	Qualys report XML templates	Creates Qualys XML and PDF report jobs through /create/was/report.	Qualys report IDs.
3	Qualys report job status	Polls /status/was/report until COMPLETE, then downloads	Detail PDF saved to assets or docs.

		detail PDF through requests.Session.	
4	Qualys XML report	Parses vulnerabilities, information-gathered results, links, rejected links, email findings, SSN/credit card findings, and severity metadata.	CSV attachments and in-memory summary data.
5	Mustache template and generated assets	Renders .tex from NEW_BIG.mustache, runs xelatex twice, removes generated auxiliary files, and optionally encrypts the PDF.	Final stakeholder PDF report.

Tracker Update

Step	Input	Processing	Output or Side Effect
1	Qualys scan schedules	Searches schedules completed since the input date and creates stakeholder objects.	Stakeholder map keyed by tag.
2	Qualys scan slices	Searches scans and matches them to stakeholders by tag, name, and cadence.	Grouped scan slices per stakeholder.
3	Scan groups and customer data	Consolidates status, no-web-service findings, manual reporting flags, FCEB flags, and Qualys errors.	Tracker item list.
4	Daily reports workbook	Inserts rows, populates columns A through T, assigns analysts from special requirements, and saves the workbook.	Updated Excel tracker.
5	DynamoDB and Qualys	Updates stakeholder scan date and app count in DynamoDB; optionally deletes	Updated metadata and possible Qualys webapp deletion.

		removed no-web-service webapps in Qualys.	
--	--	---	--

Integration Points

System	Direction	Evidence in Code	Notes for Databricks Migration
Qualys WAS API	Read and write	qualysapi.connect, qgc.request endpoints including search/was/wasscanschedule, search/was/wasscan, search/was/finding, /create/was/report, /download/was/report, /delete/was/webapp, /ignore/was/finding, and tag endpoints.	Primary data source and control plane. API calls should be isolated behind a client module with retries, timeouts, structured errors, and audit logging.
AWS DynamoDB stakeholder table	Read and write	CUSTOMER_DATA = DynamoDB(STAKEHOLDERS_TABLE_NAME, PROD_PROFILE); reads report passwords and contact metadata; writes next scheduled, last scanned, and app count attributes.	Clarify whether DynamoDB remains source of truth or is replicated into Databricks Delta tables. Maintain least-privilege IAM.
Daily reports Excel workbook	Read and write	openpyxl load_workbook; tracker rows updated and saved.	Excel workbook is workflow state. Replace with Delta tables or a governed job-state table where possible.
Special cases workbook	Read	specialCasesFilePath loaded with pandas ExcelFile.	Move analyst assignments and KEEP_NWS lists to a managed table or config dataset.

Local was_config.txt	Read	ConfigParser reads credentials, host, file paths, log path, and workbook paths from the user home directory.	Replace local workstation configuration with Databricks job parameters, environment variables, and secret scopes.
Local Docker daemon	Execution	docker.from_env and client.containers.run image was_report_creator.	Replace local Docker fanout with Databricks task orchestration or a containerized job image.
Local filesystem and mounted home directory	Read and write	Reports, assets, docs, ~/Reports, and report mailer folders are assumed.	Replace with DBFS, Unity Catalog volumes, S3, or another approved storage location.
Microsoft Teams or report mailer folder	Outbound handoff	copyReports says Teams Folder and writes report mailer ZIP; PowerAutomate trigger file logic is commented.	Document the current manual or automated handoff before replacing with a governed notification and delivery process.
Redshift	Not observed directly	No direct redshift, psycpg, sqlalchemy Redshift URL, or PostgreSQL connection use was found in backend/was static inspection.	Validate broader repo and production runtime. If downstream consumers rely on Redshift, catalog the handoff outside this export.

Legacy Dependencies and Risks

Severity	Area	Finding	Impact	Recommendation
High	Secrets and configuration	Qualys credentials and host are read from a local was_config.txt file; report passwords are	Secrets can leak through local files, process listings, shell history, or logs.	Move credentials to managed secrets, pass secrets through supported secret references, and

		passed on the container command line.		avoid command-line password arguments.
High	Runtime reproducibility	Dockerfile uses python:latest and apt-get without pinned package versions; requirements are mostly unpinned.	Builds may change unexpectedly and migration failures may be hard to reproduce.	Pin Python version, package versions, and OS dependencies. Build a controlled image for report rendering if LaTeX remains required.
High	Workflow state	Excel workbooks are used as both input queue and mutable state store.	Concurrent access, partial writes, and manual edits can corrupt workflow state or make runs non-idempotent.	Model tracker state in Delta tables or a transactional service, with Excel used only for exports if needed.
Medium	Coupling	WAS_report_creator.py combines CLI parsing, API access, XML parsing, graph generation, PDF generation, encryption, and Qualys admin actions in one large script.	Hard to test, hard to parallelize safely, and risky to migrate incrementally.	Separate API client, parser, report model, renderer, delivery, and admin operations.
Medium	Error handling	Multiple broad except blocks, sys.exit calls, polling loops, and shell calls make failures difficult to classify.	Databricks jobs may fail without structured diagnostics or retry semantics.	Use structured exceptions, bounded retries, explicit exit codes, and job-level metrics.
Medium	Local side effects	The pipeline deletes files from the report mailer folder after zipping and can delete	Unexpected data loss is possible if paths or conditions are wrong.	Add dry-run mode, explicit confirmations for destructive operations, and audit logs for

		Qualys webapps as part of tracker update.		deletes.
Low	Dependency hygiene	requirements.txt contains duplicates and likely unused packages such as pymongo for this visible code path.	Larger attack surface and slower builds.	Run dependency inventory, remove unused packages, and add vulnerability scanning.

Databricks Compatibility Considerations

The code can be migrated, but not as a simple lift-and-shift into a notebook. The current pipeline assumes a stateful workstation, local Docker, local Excel files, and local report delivery folders. Databricks compatibility will require separating data extraction, transformation, state management, report rendering, and delivery.

Use Databricks Workflows to replace the top-level Docker fanout in reporting.py.

Persist Qualys raw XML or parsed vulnerability records into Delta tables for repeatability, auditability, and downstream analytics.

Replace Excel tracker mutation with Delta-backed tracker tables, then generate Excel or CSV exports only as presentation artifacts if stakeholders still need them.

Run LaTeX/PDF rendering in a controlled job image or external rendering service if the final PDF format must stay unchanged.

Move stakeholder metadata either into a governed Databricks table replicated from DynamoDB or keep DynamoDB as source of truth behind a small integration client.

Create clear boundaries for destructive Qualys operations such as false-positive updates, webapp deletion, tag changes, and reactivation.

Recommendations

Priority	Recommendation	Rationale	Suggested Testing
1	Create a source-to-target data map for Qualys XML fields, tracker columns A through T, DynamoDB attributes, and	Databricks migration depends on knowing which fields are operational state versus report presentation.	Compare parsed outputs against a known report run using sanitized sample XML.

	generated CSV attachments.		
2	Refactor Qualys calls into a dedicated client with typed request and response handling.	Centralizes retries, timeouts, logging, authentication, and future Databricks execution behavior.	Unit test request builders and mocked API responses for success, empty, and error cases.
3	Replace local workbook queue state with a table-backed workflow state model.	Supports parallelism, idempotency, audit history, and concurrency control.	Run repeated job attempts and verify no duplicate reports, lost rows, or partial state.
4	Define a Databricks job architecture with separate extract, transform, render, and delivery tasks.	Separates scalable data work from specialized PDF rendering and delivery concerns.	Run task-level integration tests with fixture data and inspect job lineage.
5	Build a controlled report-rendering environment.	LaTeX, fonts, PDF redaction, encryption, and attachments are specialized dependencies that may not fit a standard cluster runtime.	Render a sample report and verify visual layout, attachments, encryption, and accessibility constraints.
6	Perform a security review before enabling migrated jobs.	The current code handles credentials, passwords, customer contacts, URLs, scan findings, and potentially sensitive payload data.	Run secret scanning, dependency scanning, IAM review, and log redaction tests.

Open Questions

Where is the was_dynamodb package maintained, deployed, and versioned?

Is Redshift involved elsewhere in the broader WAS reporting ecosystem, or is it only a target modernization concern from the Jira ticket?

Who owns the daily reports workbook, report mailer folder, Teams handoff, and any Power Automate flows currently used in operations?

Which report artifacts are mandatory after migration: final PDF, Qualys detail PDF attachment, CSV attachments, Excel tracker, mailer ZIP, or all of them?

Can destructive Qualys operations be separated from read-only reporting jobs during modernization?

Appendix A: Reviewed Files

backend/was/reporting.py

backend/was/wasreport/wasreport/WAS_report_creator.py

backend/was/wasreport/wasreport/requirements.txt

backend/was/wasreport/wasreport/Dockerfile

backend/was/wasreport/wasreport/NEW_BIG.mustache

backend/was/wasreport/wasreport/assets/was_report.xml

backend/was/wasreport/wasreport/assets/was_report_details.xml

backend/was/update_tracker/update_tracker/main.py

backend/was/update_tracker/update_tracker/set_up.py

backend/was/update_tracker/update_tracker/data/*.py

backend/was/update_tracker/update_tracker/models/*.py

backend/was/update_tracker/update_tracker/utils/**/*.py

Appendix B: Evidence Pointers

Finding	Evidence Pointer
Configuration and workbook paths	backend/was/reporting.py lines 30-33; backend/was/update_tracker/update_tracker/ set_up.py lines 10-36.
Docker fanout and command-line passwords	backend/was/reporting.py lines 42-94.
PDF validation, copy, archive, and zip behavior	backend/was/reporting.py lines 112-253.
Qualys connection setup	backend/was/wasreport/wasreport/ WAS_report_creator.py lines 91-96.
Qualys report creation, status polling, download, redaction, and watermarking	backend/was/wasreport/wasreport/ WAS_report_creator.py lines 609-705 and 1183-1255.
Final report generation and encryption	backend/was/wasreport/wasreport/ WAS_report_creator.py lines 997-1067 and 1443-1547.
Default report path through the CLI	backend/was/wasreport/wasreport/ WAS_report_creator.py lines 1829-1869.

Tracker workflow sequence	backend/was/update_tracker/ update_tracker/main.py lines 16-21.
Tracker workbook updates and DynamoDB updates	backend/was/update_tracker/ update_tracker/utis/tracker_operations/ update_tracker.py lines 13-145.
No direct Redshift usage observed	Static search across backend/was for redshift, psycpg, postgres, and related connection terms found no direct integration.